Viral Sample Pool Design and Deconvolution Project Progress Report

Approximation Algorithms
October 26, 2026

1 Project objective

My goal is to implement the computational approach in the chapter *Pooling Strategy for Massive Viral Sequencing* and evaluate it on viral sequence data. The main idea is to mix several samples into overlapping pools, sequence those pools, and then assign the sequences back to their original samples. A successful design should use fewer pools while keeping the samples distinguishable.

This project connects to the course because finding the minimum number of valid pools is an NP-hard optimization problem. I am implementing the chapter's greedy and tabu-search heuristics rather than claiming to find an optimal solution for every input.

2 Work completed

I have completed an initial Python implementation of the main computational stages:

Component	Current status
Sample compatibility graph	Implemented, with a titer-difference rule and optional forbidden pairs
Greedy Pool Design Algorithm (GPDA)	Implemented
Clique-cut selection with tabu search	Implemented
Exact clique-cut solver for small inputs	Added as a verification tool
FASTA input and simulated pooling	Implemented
kGEM clustering	Implemented using the chapter's likelihood model and consensus update
Generalized pool intersections and differences	Implemented
Synthetic and public HCV comparisons	Completed for the current small datasets
Verification tests and reproducible commands	Included in the repository

The code is organized into separate modules for pool design, sequence processing, public-data preparation, and benchmarking. The repository also contains input data, source attribution, checksums, saved results, and instructions for repeating the experiments.

3 How the implementation works

The compatibility graph represents which samples can be mixed. Each pool must be a clique, meaning that every pair of samples inside the pool is compatible. The pool-size limit prevents the algorithm from putting too many samples together.

Preprint.

GPDA maintains a second graph containing sample pairs that have not yet been separated. At each step, it adds a pool that separates some of these pairs. A pair is separated when exactly one of its samples belongs to the pool. The process continues until every pair has been separated. An uncovered sample is placed in a singleton pool if necessary, so every sample has a distinct, nonempty pattern of pool memberships.

The tabu-search procedure looks for a useful clique cut at each greedy step. It rewards separation of unresolved pairs and penalizes incompatible pairs inside a candidate pool. The exact solver provides a reference for checking small clique-cut instances; it does not make the overall greedy design globally optimal.

For deconvolution, ordinary sequence intersection is not sufficient because the same sample can contribute different variants to different pools. The implementation uses kGEM to cluster related sequences. A generalized intersection keeps clusters with observations from both pools. Intersections and differences are then used to separate the individual samples.

4 Data and experimental setup

I first tested the pipeline with a clearly labeled synthetic dataset containing eight artificial samples and four variants per sample. The sequences are 306 bases long and were deliberately generated with well-separated sample populations.

I then used public HCV HVR1 sequences from the dataset associated with Janiak et al. (2019). I selected the first eight chronic genotype-1 specimen files in numeric identifier order before measuring performance. These files contain 338 reconstructed haplotype records. I retained all listed variants and normalized the frequency estimates in their FASTA headers for sampling. After removing the original gap symbols, I aligned the sequences together with MAFFT 7.526. The resulting alignment has 180 columns.

For both datasets, I used the following comparison settings:

- Eight samples, with at most four samples per pool.
- All sample pairs assumed compatible for this controlled comparison.
- Eighty observations sampled per included sample per pool.
- Five sampling seeds: 91 through 95.
- Initial clustering multiplier W=2, with retries up to W=4 when a recovered sample was empty.

The HCV sequences are real, but the pools and read observations are simulated. I have not performed an experiment using laboratory-pooled specimens. The public benchmark also does not use measured specimen titers or epidemiological exclusions.

5 Results so far

Both datasets produced four pools for eight samples and 1,040 observations per run.

Measure	Synthetic dataset	Public HCV subset
Haplotype records	32	338
Alignment length	306	180
Mean read-assignment accuracy	100.00%	71.81%
Accuracy range across five seeds	100.00–100.00%	70.48–72.88%
Mean recall excluding the singleton specimen	100.00%	69.46%

Every observation received exactly one assignment in these runs. Therefore, the lower HCV accuracy reflects incorrect assignments, rather than reads being left unassigned. Accuracy here measures agreement with the source labels used to simulate the observations.

The average also hides large differences between specimens. For seed 91, only 6 of 160 observations from public specimen 80 were correctly recovered, giving 3.75% recall. Several other specimens

were recovered completely. Specimen 201 was in a singleton pool, making its recovery straightforward; this is why I also report recall excluding that specimen.

I also checked the effect of W on the public dataset using seed 91 with retries disabled:

Fixed W	Assignment accuracy	Empty recovered specimens
1	47.98%	2
2	68.27%	1
3	71.73%	0

These results show that W affects recovery in this experiment. They do not establish that increasing W always improves the method. All 48 clustering fits in the main public-data runs, including retries, reported convergence, but convergence alone did not ensure correct assignments.

6 Verification and implementation differences

The saved test run contains **19 passing tests**. The tests check pool feasibility, sample separation, the chapter's gain equations, small exact clique-cut cases, sequence clustering examples, input validation, source-file checksums, preservation of sequence content through alignment, and assignment-metric accounting.

There are several differences between executable code and the printed pseudocode that I have documented rather than treating this as an exact reproduction:

- The tabu procedure handles nonpositive gains and records only feasible cliques. It also has iteration limits and a singleton fallback to ensure progress.
- The deconvolution queue replaces both operands with their intersection and differences to avoid repeatedly retaining equivalent pool entries.
- The kGEM stopping conditions are interpreted as stopping when estimates stabilize, because the printed conditions appear reversed. Iterations are bounded.
- Numerical settings, a cap on the number of distinct initial centroids, and logarithmic likelihood calculations make the implementation executable.

The detailed source mapping and explanations are in the main README. These tests provide evidence of implementation consistency; they do not prove a global approximation guarantee or validate biological accuracy.

7 Limitations and what I learned

The main result so far is that success on the synthetic demo does not carry over to the public HCV subset. The current implementation assigns some real-sequence populations poorly, even when every recovered sample is nonempty.

The synthetic and public datasets differ in sequence length, diversity, variant counts, and frequency distributions. I cannot attribute the accuracy gap to just one of these factors. The public input consists of reconstructed haplotypes rather than raw sequencing reads, and the simulation does not include PCR bias, contamination, titer imbalance, or sequencing errors. One aligned sequence also occurs in more than one public specimen, creating ambiguity in sequence identity.

Only eight specimens from one study have been evaluated. Repeating the sampling with five seeds measures variability for those same specimens, not performance across independent patient cohorts. Also, four pools instead of eight individual samples does not by itself demonstrate a 50% reduction in sequencing costs, since the experiment does not hold total sequencing depth or laboratory costs fixed.

I have therefore completed an initial implementation and an exploratory evaluation, but I have not reproduced the chapter's published experimental accuracy.

8 Next steps

My next priority is to understand the incorrect assignments before adding more features. I plan to:

1. Trace the generalized intersections where specimens 80 and 142 lose correctly assigned observations, using the saved confusion matrices and clustering diagnostics.
2. Test sequence length, W, sampling depth, and pool-size limits separately, keeping the other settings fixed where possible.
3. Evaluate additional specimens and datasets using selection rules established before inspecting their results.
4. Review the documented interpretations of the chapter's pseudocode, especially queue refinement and the consensus update, before comparing any alternative versions.

These are planned experiments, not completed improvements. I will report changes in accuracy together with any additional departures from the chapter.

9 Reproducing the current work

Run these commands from the repository root:

```
python3 -m viral_pooling demo
python3 -m viral_pooling.benchmark
python3 -m unittest discover -s tests -v
```

The committed public alignment allows the comparison to run without downloading data or installing MAFFT. MAFFT is needed only to regenerate that alignment.

10 References and supporting files

1. Skums et al. (2016). *Pooling Strategy for Massive Viral Sequencing*. Chapter 3 in *Computational Methods for Next Generation Sequencing Data Analysis*, pp. 57–83.
2. Janiak et al. (2019). *Hepatitis C virus (HCV) genotype 1b displays higher genetic variability of hypervariable region 1 (HVR1) than genotype 3*. Scientific Reports.
3. Caraballo Cortes, K. (2019). Public HCV HVR1 sequence dataset. Zenodo, DOI 10.5281/zenodo.3266795, CC BY 4.0.

- Implementation and reproduction notes
- Full comparison report
- Machine-readable comparison results
- Public-data provenance and preparation
- Saved test output